SVM. Часть 2. Решение задачи: primal (прямая постановка)

0. Что мы делаем в этой части

В части 1 мы пришли к такой задаче: найти $\mathbf{w}$ и b, для которых зазор $2/\|\mathbf{w}\|$ максимален, и при этом все точки стоят по свою сторону: $y_i(\mathbf{w} \cdot x_i + b) \geq 1$.

Теперь нужно эту задачу **решить на компьютере**. Мы будем искать $\mathbf{w}$ и b градиентным спуском: берём какие-то стартовые значения и маленькими шагами двигаем их туда, где «ошибка» меньше.

Но у градиентного спуска есть требование: ему нужна **одна функция потерь**, которую мы минимизируем. А у нас пока не функция, а условие-ограничение. Поэтому сначала превращаем задачу в функцию потерь. Это и есть **primal** (прямой) способ: мы работаем прямо с $\mathbf{w}$ и b.

Обозначения те же, что в части 1:

- x_i — признаки точки i, y_i — её метка ($+1$ или -1);
- $\mathbf{w} \cdot x_i + b$ — «оценка» точки: чем больше по модулю, тем увереннее ответ;
- N — число точек в выборке.

---

1. Две цели вместо одной

SVM хочет сразу двух вещей:

1. **Цель 1.** Чтобы все точки были классифицированы правильно и лежали вне зазора.
2. **Цель 2.** Чтобы зазор был как можно шире.

Для каждой цели придумаем свою функцию потерь, а потом сложим.

1.1. Цель 1: правильная классификация

Условие из части 1 (шаг 4.5):

$$y_i \left(\mathbf{w} \cdot x_i + b\right) \geq 1$$

Назовём число $m_i = y_i(\mathbf{w} \cdot x_i + b)$ **запасом точки** i. Оно показывает, как далеко точка заехала в «свою» сторону. Для каждой точки возможны три ситуации:

Запас m_i	Где точка	Хорошо ли это
$m_i \geq 1$	на своей стороне, вне зазора (или ровно на его границе)	да, всё в порядке
$0 < m_i < 1$	на своей стороне, но **внутри зазора**	нет, слишком близко к линии
$m_i \leq 0$	на **чужой** стороне или на самой линии	нет, это ошибка классификации

Нам нужно, чтобы у всех точек было $m_i \geq 1$. Всё, что меньше единицы, — это «плохо», и мы хотим за это штрафовать.

1.2. Hinge loss (петлевая функция потерь)

Штраф для одной точки сделаем таким:

- если $m_i \geq 1$, штраф равен 0 (всё хорошо, наказывать не за что);
- если $m_i < 1$, штраф равен $1 - m_i$ (чем сильнее не дотянули до единицы, тем больше штраф).

Одной строкой это записывается так:

$$\text{штраф}_i = \max\bigl(0,\ 1 - y_i(\mathbf{w} \cdot x_i + b)\bigr)$$

Здесь $\max(0, \ldots)$ значит «возьми большее из двух»: если выражение внутри отрицательное, получится 0.

Пример. Пусть у трёх точек запасы $m = 3$, 0.6 и -0.5.

- $m = 3$: штраф $\max(0,\ 1 - 3) = \max(0,\ -2) = 0$.
- $m = 0.6$: штраф $\max(0,\ 1 - 0.6) = 0.4$ (точка внутри зазора).
- $m = -0.5$: штраф $\max(0,\ 1 + 0.5) = 1.5$ (точка на чужой стороне, штраф самый большой).

Для всей выборки берём среднее по точкам:

$$\text{HingeLoss} = \frac{1}{N} \sum_{i=1}^{N} \max\bigl(0,\ 1 - y_i(\mathbf{w} \cdot x_i + b)\bigr)$$

Чем больше точек лежит на своей стороне **вне** зазора, тем меньше эта величина. Идеальное значение — ноль: все точки с запасом ≥ 1.

1.3. Цель 2: максимальный зазор

Ширина зазора равна $2/\|\mathbf{w}\|$ (часть 1, шаг 4.6). Чтобы её увеличить, нужно **уменьшать длину** $\mathbf{w}$. Как и в части 1, вместо $\|\mathbf{w}\|$ берём квадрат $\|\mathbf{w}\|^2$.

Нам без разницы, что минимизировать: длину или её квадрат. Само значение нас не интересует, оно может идти хоть от 10 до 1, хоть от 1000 до 10. Важны только те $\mathbf{w}$, которые мы получим в конце, а у $\|\mathbf{w}\|$ и $\|\mathbf{w}\|^2$ они одни и те же: минимум достигается в одной и той же точке. Квадрат берём лишь потому, что без корня считать проще.

Это и есть вторая часть потерь:

$$\|\mathbf{w}\|^2 = w_1^2 + w_2^2 + \cdots + w_n^2 \;\to\; \min$$

> Замечание про запись. В части 1 мы писали $\frac{1}{2}\|\mathbf{w}\|^2$, здесь пишем без $\frac{1}{2}$. Это не меняет ответ: если умножить функцию на положительное число, её минимум остаётся в той же точке. Зато производная получится чуть проще: $2\mathbf{w}$ без лишних дробей.

1.4. Итоговая функция потерь

Складываем две цели:

$$L = \underbrace{\|\mathbf{w}\|^2}_{\text{хотим широкий зазор}} + \underbrace{\frac{1}{N}\sum_{i=1}^{N} \max\bigl(0,\; 1 - y_i(\mathbf{w}\cdot x_i + b)\bigr)}_{\text{хотим правильные ответы вне зазора}}$$

Конкретное число L нам не важно: оно зависит от данных и от стартовых $\mathbf{w}$ и b, и может идти хоть от 1000 до 10, хоть от 10 до 1. Важно другое: при каких $\mathbf{w}$ и b оно получается наименьшим. Именно их мы ищем, шаг за шагом меняя $\mathbf{w}$ и b в сторону уменьшения L.

---

2. Почему обычный градиентный спуск не подходит

Градиентному спуску нужна производная функции потерь. Посмотрим на штраф одной точки как на функцию от запаса m:

- при $m > 1$ он равен 0 (график идёт по оси, производная 0);
- при $m < 1$ он равен $1 - m$ (наклонная прямая, производная -1);
- ровно при $m = 1$ график **ломается** под углом, там две разные «производные» слева и справа, и однозначной нет.

Такая точка излома и есть «петля» (hinge), откуда название.

Выход простой. В точке излома мы можем взять любую из двух соседних производных, и спуск всё равно будет работать. Метод, который так делает, называется **субградиентным спуском**. Нам достаточно договориться:

- если $m_i \geq 1$, берём производную первой ветки (штраф равен нулю);
- если $m_i < 1$, берём производную второй ветки (штраф $1 - m_i$).

Так излом нас больше не беспокоит. Дальше просто называем это «градиент».

---

3. Градиент для одной точки

Метод работает **по одной точке за раз**: берём точку x_i, смотрим, выполняется ли для неё условие, и считаем градиент. Нужны две производные: по $\mathbf{w}$ и по b.

3.1. Случай 1: $y_i(\mathbf{w} \cdot x_i + b) \geq 1$

Точка в порядке, штраф за ошибку равен нулю. Остаётся только вторая цель, $\|\mathbf{w}\|^2$. Её производная равна $2\mathbf{w}$:

$$\nabla_{\mathbf{w}} = 2\mathbf{w} \qquad\qquad \nabla_b = 0$$

Смысл: точка ничего не просит, и мы просто чуть-чуть уменьшаем $\mathbf{w}$, расширяя зазор. Для b градиент нулевой, потому что b не входит в $\|\mathbf{w}\|^2$.

3.2. Случай 2: $y_i(\mathbf{w} \cdot x_i + b) < 1$

Точка внутри зазора или на чужой стороне. Теперь к $2\mathbf{w}$ добавляется производная штрафа $1 - y_i(\mathbf{w} \cdot x_i + b)$. Считаем её по частям:

- по $\mathbf{w}$: $1 - y_i(\mathbf{w} \cdot x_i + b)$ — это линейная функция, а производная $\mathbf{w} \cdot x_i$ по $\mathbf{w}$ равна x_i, поэтому производная всего штрафа равна $-y_i\, x_i$;
- по b: производная b равна 1, поэтому для штрафа получается $-y_i$.

Итого:

$$\nabla_{\mathbf{w}} = 2\mathbf{w} - y_i\, x_i \qquad\qquad \nabla_b = -\, y_i$$

3.3. Что значат эти формулы по-человечески

Шаг делается в сторону **минус** градиента, то есть к $-\nabla$. Посмотрим, что получается для проблемной точки:

$$\mathbf{w}_{\text{новый}} = \mathbf{w} - \alpha\,(2\mathbf{w} - y_i x_i) = \mathbf{w} - 2\alpha\mathbf{w} + \alpha\, y_i\, x_i$$

Слагаемое $+\alpha\, y_i\, x_i$ **добавляет** к $\mathbf{w}$ саму точку, взятую со своим знаком:

- у точки класса $+1$ вектор $\mathbf{w}$ придвигается к x_i, оценка $\mathbf{w} \cdot x_i$ растёт;
- у точки класса -1 вектор $\mathbf{w}$ отодвигается от x_i, оценка $\mathbf{w} \cdot x_i$ падает.

В обоих случаях запас m_i увеличивается, то есть точку «подталкивают» в правильную сторону. Слагаемое $-2\alpha\mathbf{w}$ при этом слегка сжимает $\mathbf{w}$ (расширяет зазор). Для b сдвиг $+\alpha\, y_i$ делает то же самое: двигает линию так, чтобы точка получила больший запас.

---

4. Шаг градиентного спуска

После того как посчитан градиент для точки, делаем шаг в противоположную сторону:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha\, \nabla_{\mathbf{w}} \qquad\qquad b \leftarrow b - \alpha\, \nabla_b$$

Здесь α — размер шага (learning rate). Это число, которое вы задаёте сами:

- слишком большое α: линия «прыгает» и может не успокоиться;
- слишком маленькое α: линия двигается очень медленно, нужно много итераций.

Обновление происходит **после каждой точки**, а не один раз за проход по всей выборке. Почему при этом пропадает множитель $1/N$? Потому что L — это **среднее по точкам** от выражения $\|\mathbf{w}\|^2 + \text{штраф}_i$. На каждом шаге мы берём одно такое слагаемое (для точки i), и у него $1/N$ нет. Градиент этого слагаемого — как раз формулы из шага 3. Полное L со средним нужно только тогда, когда мы хотим посчитать значение для лога.

---

5. Алгоритм обучения (псевдокод)

```
Вход:  X (признаки), y (метки 0/1), n_iter (число проходов), alpha
(размер шага)

1. Перевести метки: 0 -> -1, 1 -> +1

2. Задать начальные значения:
       w = вектор из единиц (длина = число признаков)
       b = 1

3. Повторить n_iter раз:                      # один проход = одна эпоха

       3.1. Для каждой точки (x_i, y_i) из выборки:

              а) посчитать запас:   m = y_i * (w · x_i + b)

              б) если m >= 1:                 # точка в порядке
                     grad_w = 2 * w
                     grad_b = 0
                 иначе:                       # точка в зазоре или ошибка
                     grad_w = 2 * w - y_i * x_i
                     grad_b = - y_i

              в) сделать шаг:
                     w = w - alpha * grad_w
                     b = b - alpha * grad_b

       3.2. (для лога) посчитать потери на всей выборке:
                 L = (w · w) + среднее( max(0, 1 - y * (w · x + b)) )
              и вывести номер итерации и L, если пора

4. Вернуть w и b
```

Предсказание для новой точки:

```
оценка = w · x + b
если оценка > 0:  класс +1  (в исходных метках: 1)
иначе:            класс -1  (в исходных метках: 0)
```

---

6. Числовой пример: один проход по двум точкам

Возьмём две точки из части 1 (признаки двумерные):

- точка 1: $x_1 = (4,\ -2)$, метка $y_1 = +1$;
- точка 2: $x_2 = (-2,\ 0)$, метка $y_2 = -1$.

Старт: $\mathbf{w} = (1,\ 1)$, $b = 1$, размер шага $\alpha = 0.1$.

Потери до обучения.

- Запас точки 1: $1 \cdot (4 - 2 + 1) = 3$, штраф 0.
- Запас точки 2: $-1 \cdot (-2 + 0 + 1) = 1$, штраф 0.
- Длина: $\|\mathbf{w}\|^2 = 1 + 1 = 2$.

$$L_{\text{старт}} = 2 + \frac{0 + 0}{2} = 2.0$$

Точка 1. Запас $m = 3 \geq 1$, значит случай 1:

$$\nabla_{\mathbf{w}} = 2\mathbf{w} = (2,\ 2) \qquad \nabla_b = 0$$

$$\mathbf{w} = (1,\ 1) - 0.1 \cdot (2,\ 2) = (0.8,\ 0.8) \qquad b = 1$$

Точка 2. С новым $\mathbf{w}$ запас равен $-1 \cdot (0.8 \cdot (-2) + 0.8 \cdot 0 + 1) = -1 \cdot (-0.6) = 0.6 < 1$. Точка внутри зазора, значит случай 2:

$$\nabla_{\mathbf{w}} = 2\mathbf{w} - y_2 x_2 = (1.6,\ 1.6) - (-1) \cdot (-2,\ 0) = (1.6,\ 1.6) - (2,\ 0) = (-0.4,\ 1.6)$$

$$\nabla_b = -y_2 = +1$$

$$\mathbf{w} = (0.8,\ 0.8) - 0.1 \cdot (-0.4,\ 1.6) = (0.84,\ 0.64) \qquad b = 1 - 0.1 \cdot 1 = 0.9$$

Проверка. Запас точки 2 был 0.6, а стал $-1 \cdot (0.84 \cdot (-2) + 0.9) = -1 \cdot (-0.78) = 0.78$. Он вырос, точка отодвинулась от линии.

Потери после прохода.

- Запас точки 1: $0.84 \cdot 4 - 0.64 \cdot 2 + 0.9 = 2.98$, штраф 0.
- Запас точки 2: 0.78, штраф $1 - 0.78 = 0.22$.
- Длина: $\|\mathbf{w}\|^2 = 0.84^2 + 0.64^2 = 0.7056 + 0.4096 = 1.1152$.

$$L = 1.1152 + \frac{0 + 0.22}{2} = 1.2252$$

Потери упали с 2.0 до 1.2252 уже за один проход: $\mathbf{w}$ и b сдвинулись в сторону меньшего L. Так и должно быть: в логе L в целом уменьшается от итерации к итерации. Ровно уменьшаться на каждом шаге оно не обязано, потому что мы двигаемся по одной точке за раз.

---

7. Итог

1. Условие $y_i(\mathbf{w} \cdot x_i + b) \geq 1$ превращаем в штраф hinge loss: ноль для хороших точек и $1 - m_i$ для плохих.
2. К нему добавляем $\|\mathbf{w}\|^2$, чтобы зазор был широким.
3. Функция L ломается в одной точке, поэтому берём «градиент по веткам»: $2\mathbf{w}$ для хороших точек и $2\mathbf{w} - y_i x_i$ (по b: $-y_i$) для плохих.
4. Идём по точкам по одной и после каждой делаем шаг $-\alpha$·градиент.
5. Повторяем n раз, следим, чтобы L падал.

В части 3 мы решим ту же задачу с другой стороны, через **dual** (двойственную) постановку.